

HITO 2 DEL 2º TRIMESTRE DE Bases de datos

Ignacio Arias

CampusFP

FECHA LIMITE 13 de febrero

Índice

Programación con MySQL	2
1. Crea una base de datos llamada triángulos.	2
2. Crea un procedimiento almacenado dentro de la base de datos triángulos.....	2
3. Crea una función PL/SQL que reciba retorna una cadena indicando si el triángulo es Equilátero, Isósceles o Escaleno.....	3
4. Crea una función PL/SQL que retorne el perímetro de dicho triángulo.	4
5. Utiliza las dos funciones anteriores en una sentencia SELECT.....	4
Programación con PostgreSQL.....	5
1. Crea una base de datos llamada triángulos.	5
2. Crea un procedimiento almacenado dentro de la base de datos triángulos:.....	6
3. Crea una función PL/PGSQL que retorna una cadena indicando si el triángulo es Equilátero, Isósceles o Escaleno.....	7
4. Crea una función PL/PGSQL que retorne el perímetro de dicho triángulo.....	7
5. Utiliza las dos funciones anteriores en una sentencia SELECT para obtener una relación de triángulos.....	8
Bibliografía	9

Programación con MySQL

1. Crea una base de datos llamada triángulos.

```
create database Triangulos;  
use Triangulos;
```

2. Crea un procedimiento almacenado dentro de la base de datos triángulos

```
CREATE PROCEDURE Crear_Triangulos ()  
BEGIN  
    drop table if exists lados;  
    create table lados(  
        id_triangulo int auto_increment primary key,  
        lado1 int,  
        lado2 int,  
        lado3 int  
    );  
END
```

```
CREATE PROCEDURE Crear_Triangulos ()  
BEGIN  
    drop table if exists triangulo;  
    create table triangulo(  
        id_triangulo int auto_increment primary key,  
        lado1 int,  
        lado2 int,  
        lado3 int  
    );  
  
    while i<20 do  
        insert into triangulo(lado1, lado2, lado3) values  
        (floor(1 + rand()*5), floor(1+rand()*5), floor(1 + rand()* 5 ));  
        set i = i +1;  
    end while;  
END
```

```
select * from triangulo;
call Crear_Triangulos();
```

id_triangulo	lado1	lado2	lado3
1	1	3	5
2	5	2	5
3	1	1	1
4	2	1	1
5	3	2	5
6	1	4	2
7	2	5	1
8	5	5	3
9	1	3	2
10	4	4	2
11	4	5	5
12	4	4	2
13	4	2	3

3. Crea una función PL/SQL que reciba retorna una cadena indicando si el triángulo es Equilátero, Isósceles o Escaleno.

```
CREATE DEFINER='root'@'localhost' FUNCTION `tipo_triangulo`(lado1 int, lado2 int, lado3 int) RETURNS varchar(20) CHARSET utf8mb4
DETERMINISTIC
BEGIN
    declare tipo_triangulo varchar(20);

    if lado1 = lado2 and lado1 = lado3 then
        set tipo_triangulo = "Equilatero";
    elseif lado1 = lado2 or lado1 = lado3 or lado2 = lado3 then
        set tipo_triangulo = "Isosceles";
    else
        set tipo_triangulo = "Escaleno";
    end if;

    RETURN tipo_triangulo;
END
```

```
select id_triangulo, tipo_triangulo(lado1,lado2,lado3) as tipo_triangulo
from triangulo;
```

id_triángulo	tipo_triángulo
1	Escaleno
2	Isosceles
3	Equilatero
4	Isosceles
5	Escaleno
6	Escaleno
7	Escaleno
8	Isosceles
9	Escaleno
10	Isosceles
11	Isosceles
12	Isosceles
13	Escaleno

4. Crea una función PL/SQL que retorne el perímetro de dicho triángulo.

```
CREATE DEFINER='root'@'localhost' FUNCTION `Perimetro`(lado1 int, lado2 int, lado3 int) RETURNS int
    DETERMINISTIC
BEGIN
    RETURN (lado1+lado2+lado3);
END
```

```
select id_triángulo, Perimetro(lado1,lado2,lado3) as perimetro_triángulo
from triangulo;
```

id_triángulo	perimetro_triángulo
1	9
2	12
3	3
4	4
5	10
6	7
7	8
8	13
9	6
10	10
11	14
12	10
13	9

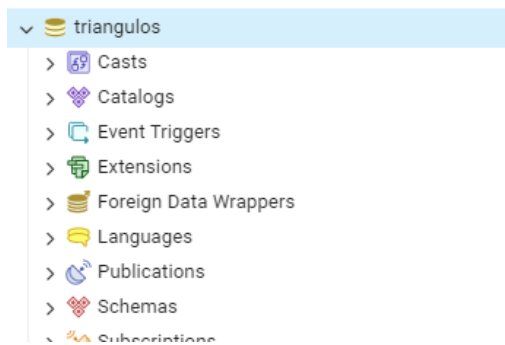
5. Utiliza las dos funciones anteriores en una sentencia SELECT

```
select id_triángulo, tipo_triángulo(lado1,lado2,lado3) as tipo_triángulo,
    Perimetro(lado1,lado2,lado3) as perimetro_triángulo from triangulo;
```

id_triangulo	tipo_triangulo	perimetro_triangulo
1	Escaleno	9
2	Isosceles	12
3	Equilatero	3
4	Isosceles	4
5	Escaleno	10
6	Escaleno	7
7	Escaleno	8
8	Isosceles	13
9	Escaleno	6
10	Isosceles	10
11	Isosceles	14
12	Isosceles	10
13	Escaleno	8

Programación con PostgreSQL

1. Crea una base de datos llamada triángulos.



2. Crea un procedimiento almacenado dentro de la base de datos triángulos:

--Crear procedimiento--

```
CREATE OR REPLACE PROCEDURE crear_triángulos()
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
-- Si la tabla ya existe, se elimina
```

```
DROP TABLE IF EXISTS triangulo;
```

```
-- Se crea la tabla triangulo con un único campo de tipo array
```

```
CREATE TABLE triangulo (
```

```
    id_triángulo serial,
```

```
    lados INTEGER[]
```

```
);
```

```
-- Insertar 20 filas con valores aleatorios entre 1 y 5 para cada lado
```

```
FOR i IN 1..20 LOOP
```

```
    INSERT INTO triangulo (lados)
```

```
    VALUES (ARRAY[
```

```
        (floor(random()*5)+1)::int,
```

```
        (floor(random()*5)+1)::int,
```

```
        (floor(random()*5)+1)::int
```

```
    ]);
```

```
END LOOP;
```

```
END;
```

```
$$;
```

```
call crear_triángulos();
```

```
select * from triangulo;
```

	id_triángulo integer	lados integer[]
1	1	{1,2,3}
2	2	{2,5,2}
3	3	{1,5,1}
4	4	{4,3,3}
5	5	{3,2,3}
6	6	{4,2,1}
7	7	{4,4,1}

3. Crea una función PL/PGSQL que retorna una cadena indicando si el triángulo es Equilátero, Isósceles o Escaleno.

```
--Funcion tipo de triangulo--
create or replace function tipo_triangulo(lados integer[])
returns text
language plpgsql
as $$
begin
    if lados[1] = lados[2] and lados[1] = lados[3] then
        return "Equilatero";
    elseif lados[1] = lados[2] or lados[2] = lados[3] or lados[1] = lados[3] then
        return "Isosceles";
    else
        return "Escaleno";
    end if;
end;
$$
```

```
select id_triangulo, tipo_triangulo(lados) as tipos from triangulo;
```

id_triangulo integer	tipos text
1	Escaleno
2	Isosceles
3	Isosceles
4	Isosceles
5	Isosceles
6	Escaleno
7	Isosceles

4. Crea una función PL/PGSQL que retorne el perímetro de dicho triángulo.

```
--Funcion perimetro--
create or replace function perimetro_triangulo(lados integer[])
returns integer
language plpgsql
as $$
begin
    return lados[1] + lados[2] + lados[3];
end;
$$
```

```
select id_triangulo, perimetro_triangulo(lados) as perimetro from triangulo;
```

id_triángulo integer	perímetro integer
1	6
2	9
3	7
4	10
5	8
6	7
7	9

5. Utiliza las dos funciones anteriores en una sentencia SELECT para obtener una relación de triángulos

```
select id_triángulo, tipo_triángulo(lados) as tipos,
perímetro_triángulo(lados) as perímetro from triángulo;
```

id_triángulo integer	tipos text	perímetro integer
1	Escaleno	6
2	Isosceles	9
3	Isosceles	7
4	Isosceles	10
5	Isosceles	8
6	Escaleno	7
7	Isosceles	9

Bibliografía

How to generate a random number in A range. (s. f.). Neon. Recuperado 10 de febrero de 2025, de <https://neon.tech/postgresql/postgresql-tutorial/postgresql-random-range>

PostgreSQL Array. (s. f.). Neon. Recuperado 10 de febrero de 2025, de <https://neon.tech/postgresql/postgresql-tutorial/postgresql-array>

PostgreSQL tutorial. (s. f.). W3schools.com. Recuperado 10 de febrero de 2025, de <https://www.w3schools.com/postgresql/>

SQL stored procedures. (s. f.). W3schools.com. Recuperado 10 de febrero de 2025, de https://www.w3schools.com/sql/sql_stored_procedures.asp